

Singleton Pattern

- The Singleton pattern ensures a class has **only one instance** throughout the lifetime of the application and provides a **global access point** to it.
- It is one of the simplest GoF patterns but also one of the most abused – understanding when it is appropriate is as important as knowing how to implement it correctly.

Bad Example

// Thread-unsafe singleton: two threads can both see _instance == null and each create their own instance. Double-check locking done incorrectly.

```
sealed class ConfigurationManager
{
    private static ConfigurationManager? _instance;
    private ConfigurationManager()
    {
        Console.WriteLine("Loading configuration from disk...");
    }
    public static ConfigurationManager Instance
    {
        get
        {
            if (_instance == null) // ← race condition here
                _instance = new ConfigurationManager();
            return _instance;
        }
    }
    public string GetSetting(string key) => $"value-of-{key}";
}
```

Approach 1: Lazy<T> – recommended

```
sealed class AppConfiguration
{
    private static readonly Lazy<AppConfiguration> _lazy = new(() => new AppConfiguration());
    public static AppConfiguration Instance => _lazy.Value;
    // A GUID assigned once at construction – proves the same instance is always returned.
    public Guid InstanceId { get; } = Guid.NewGuid();
    private readonly Dictionary<string, string> _settings = new();
    private AppConfiguration()
    {
        // Simulate loading from disk / environment
        _settings["DatabaseUrl"] = "Server=prod-db;Database=app;";
        _settings["MaxConnections"] = "100";
        _settings["LogLevel"] = "Warning";
        Console.WriteLine($"  AppConfiguration initialised (once). InstanceId = {InstanceId}");
    }
    public string Get(string key) => _settings.TryGetValue(key, out var value) ? value : $"<missing:{key}>";
}
```

Approach 2: Static readonly – eager, also thread-safe

```
sealed class MetricsCollector
{
    public static readonly MetricsCollector Instance = new();
    // A GUID assigned once at construction – proves the same instance is always returned.
    public Guid InstanceId { get; } = Guid.NewGuid();
    private int _requestCount;
    private MetricsCollector()
    {
        Console.WriteLine($" MetricsCollector initialised (once). InstanceId = {InstanceId}");
    }
    public void RecordRequest() => Interlocked.Increment(ref _requestCount);
    public int TotalRequests => _requestCount;
}
```

Approach 3: Double-check lock (for reference only)

```
sealed class LegacySingleton
{
    private static LegacySingleton? _instance;
    private static readonly object _lock = new();
    public Guid InstanceId { get; } = Guid.NewGuid();
    private LegacySingleton()
    {
        Console.WriteLine($" LegacySingleton initialised (once). InstanceId = {InstanceId}");
    }
    public static LegacySingleton Instance
    {
        get
        {
            if (_instance is null)
            {
                lock (_lock)
                {
                    _instance ??= new LegacySingleton();
                }
            }
            return _instance;
        }
    }
}
```

Approach 4: DI – recommended

```
// In production code, prefer registering the singleton with the DI container rather than using static access.  
// This avoids hidden global-state coupling and makes dependencies explicit.  
// Example with Microsoft.Extensions.DependencyInjection:  
var builder = WebApplication.CreateBuilder(args);  
  
builder.Services.AddControllers();  
  
//Singleton for the repository, in a real application this would be a concrete implementation of the repository interface  
builder.Services.AddSingleton<IPersonRepository, IPersonRepository>();  
  
builder.Services.AddOpenApi();  
  
var app = builder.Build();  
  
// Configure the HTTP request pipeline.  
app.UseHttpsRedirection();  
  
app.UseAuthorization();  
  
app.MapControllers();  
  
app.Run();
```